Как работает пайплайн

От разбора APK до поиска устойчивого K: распознавание русской речи, диаризация и локальная LLM на устройстве

Android-приложение **com.example.baseapp** · отчёт от 8 ноября 2026

Коротко. Приложение полностью офлайн: GigaAM v3 распознаёт русскую речь, pyannote + ERes2Net делят её на спикеров, каждому слову присваивается номер говорящего, реплики склеиваются в диалог, локальная Qwen2.5 1.5B расставляет пунктуацию. Наружу отдаётся JSON `{transcript:[...]}` с целочисленным `speaker_id`.

Проверено на устройстве: iQOO 15R (Android 16, arm64), диалог 56 с — 2 спикера, 119 слов, 4 реплики, корректная пунктуация от LLM.

Содержание

1. Общая схема пайплайна
2. Что внутри APK
3. Декодирование аудио
4. Нарезка на сегменты
5. Распознавание речи: GigaAM v3
6. Диаризация и автоматическое число спикеров
7. Назначение спикера каждому слову
8. Склейка реплик и JSON
9. Локальная LLM и системный промпт
10. Метрики и результаты
11. Ограничения
12. Где что лежит в коде

1. Общая схема

Пайплайн линейный, но две ветки — распознавание и диаризация — идут независимо по одному и тому же буферу 16 кГц и встречаются на шаге назначения спикеров словам. Это важно: ASR не нужно ждать диаризацию, а диаризация не зависит от текста.

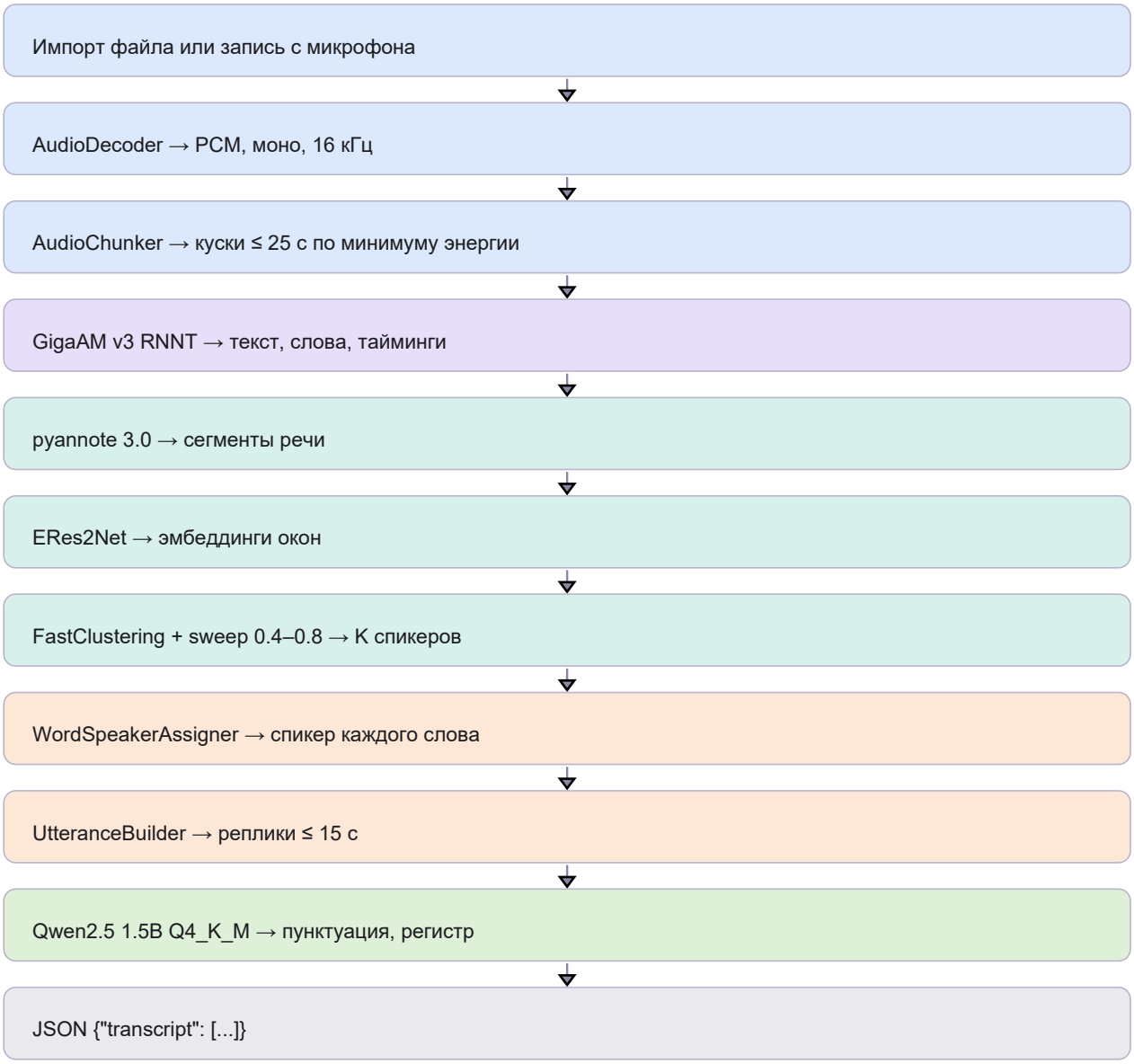


Рис. 1. Путь одного файла: от импорта до JSON.

Прогресс в интерфейсе привязан к этим шагам: 2 % — старт, 10 % — аудио декодировано, 70 % — ASR готов, 92 % — диаризация готова, 100 % — LLM закончила правку. Поэтому полоса прогресса «замирает» на 92 % дольше всего: именно там работает 1.5-миллиардная модель.

2. Что внутри APK

Приложение собрано под `minSdk 24` и содержит две ABI: `arm64-v8a` и `x86_64`. Готовый debug-APK занимает 93 863 438 байт. Внутри — 14 dex-файлов и 16 нативных библиотек; моделей в APK нет.

Ключевые нативные библиотеки

Библиотека	Размер	Зачем

libonnxruntime.so	21,2 МБ	инференс ONNX-моделей
libsherpa-onnx-jni.so	4,4 МБ	JNI-мост Kotlin → C++ sherpa-onnx
libsherpa-onnx-c-api.so	4,1 МБ	C-API sherpa-onnx
libllama.so + libggml*.so	4,1 МБ	llama.cpp для GGUF
libllama_jni.so	0,04 МБ	JNI-мост к llama.cpp
libc++_shared.so	1,2 МБ	рантайм C++

Цепочка вызовов для ASR и диаризации: Kotlin-обёртка sherpa-onnx → JNI → C++ sherpa-onnx → ONNX Runtime → int8-граф. Для LLM: Kotlin-адаптер `LlamaCppLocalLlm` → AAR `llama-android` → llama.cpp на CPU/NEON. Ни облака, ни сетевых вызовов в момент распознавания нет.

Модели: скачиваются один раз

Что	Путь в filesDir	Размер
GigaAM v3 (RNNT)	models/gigaam-v3/	229 МБ
Сегментатор pyannote 3.0	models/speaker-diarization/model.onnx	5,7 МБ
Эмбеддинги ERes2Net	models/speaker-diarization/…eres2net….onnx	37,8 МБ
Qwen2.5 1.5B Q4_K_M	models/llm/qwen2.5-1.5b-instruct-q4_k_m.gguf	1,04 ГБ

Модели вынесены из APK осознанно: 229 МБ ASR-графа и 1,04 ГБ LLM не проходят ни по лимиту магазина, ни по здравому смыслу обновлений. Качаются при первом запуске (LLM — автоматически), проверяются по размеру, а GGUF — ещё и по SHA-256. Если файла нет, приложение работает, но без LLM-пунктуации.

3. Декодирование аудио

На вход принимаются WAV, MP3, MP4/M4A/AAC и всё, что умеет системный MediaCodec. Результат всегда один: `FloatArray` в диапазоне −1…1, моно, 16 кГц.

WAV читается вручную

Парсер RIFF проходит чанки, читает `fmt ` и `data`, поддерживает PCM 8/16/24/32 бита, IEEE float 32/64 и `WAVE_FORMAT_EXTENSIBLE` (реальный формат берётся из SubFormat GUID). Каналы усредняются в моно. Такой путь не зависит от кодеков устройства и работает даже на «сырых» файлах из датасета.

MP3 / MP4 — через MediaCodec

MediaExtractor выбирает первую дорожку с MIME `audio/*` (видео игнорируется), MediaCodec декодирует её в PCM. Промежуточный WAV не создаётся. Поддержаны PCM16, PCM float и PCM8 на выходе декодера.

Ресемплинг

Линейная интерполяция к 16 кГц. Для речи этого достаточно, а стоимость нулевая. Ограничение честно записано в LIMITATIONS: на музыке и очень высоких частотах возможны алиасы; для ASR это не критично.

4. Нарезка на сегменты

Офлайн-модели из лидерборда плохо переносят длинный вход, а Whisper в sherpa-onnx вообще смотрит только первые 30 секунд. Поэтому аудио режется на куски не длиннее 25 секунд.

Как выбирается граница

- Сигнал делится на 20-мс рамки, для каждой считается RMS-энергия.
- Кандидаты на разрез — вторая половина куска (не меньше 50 % длины).
- Выбирается самая тихая рамка: обычно это пауза между словами.
- Полностью тихие куски пропускаются: пик меньше 1e-4.

Смещение куска сохраняется, поэтому все тайминги слов пересчитываются в абсолютное время файла ещё до склейки.

5. Распознавание речи: GigaAM v3

Модель: **GigaAM v3**, русский, MIT, ~229 МБ. Это RNNT-трансдьюсер NeMo, сконвертированный в ONNX. В APK-пайплайне используются `encoder.int8.onnx` (224,6 МБ), `decoder.onnx` и `joiner.onnx`.

Конфигурация распознавателя

Параметр	Значение
Тип модели	nemo_transducer
Признаки	fbank, 16 кГц, 80 измерений
Декодирование	greedy_search (без beam search)

Потоки	4 (numThreads)
Кусок	≤ 25 с

Greedy-декодер на каждом кадре берёт argmax по джойнеру: encoder даёт акустическое представление, decoder — состояние предсказателя, joiner смешивает их и выдаёт распределение по токенам. Blank-токены пропускаются, повторы схлопываются. Для русской речи это даёт WER 2,44 % при RTF 0,043 — быстрее реального времени в 23 раза.

Из токенов в слова

Модели отдают токены по-разному: GigaAM — по символу, Zipformer и Parakeet — SentencePiece с маркером ▁, Whisper — BPE. Модуль `WordTimestamps` приводит всё к словам:

- ▁ (U+2581) и Ġ считаются пробелом;
- служебные токены вида `<blk>`, `<sos/eos>` отбрасываются;
- пунктуация приклеивается к предыдущему слову, а не становится отдельным словом;
- если тайминги токенов пришли нулевые, длительность куска делится пропорционально длине токенов;
- если у токена нет длительности, конец оценивается по следующему токену или как 60 мс на символ.

На выходе — список `WordTimestamp(word, start, end)` в секундах от начала файла. Именно эти границы потом сравниваются с сегментами диаризации.

6. Диаризация: полная цепочка

Задача диаризации — разметить, **кто и когда** говорил. Формально на вход подаётся моно-сигнал 16 кГц, на выходе нужен список интервалов (`start, end, speaker`). Сложность в том, что число спикеров заранее неизвестно, речь может перекрываться, а громкость и длина реплик различаются в разы.

В приложении используется связка sherpa-onnx: сегментатор pyannote 3.0, экстрактор эмбеддингов ERes2Net base и быстрая кластеризация. В исследовательском бенчмарке — собственный пайплайн с complete-linkage AHC и gap-статистикой. Ниже разобраны оба варианта и объяснено, почему на телефоне выбран перебор порогов.



Рис. 2. Диаризация: от окон сегментатора до спикеров и границ реплик.

6.1. Сегментация речи: pyannote 3.0

Модель работает окнами по **10 секунд** со сдвигом 50 % (5 секунд). Внутри окна она предсказывает не «спикера», а класс активности: powerset-кодирование из **7 классов** описывает все комбинации до трёх одновременных голосов, включая пустой кадр. Именно поэтому модель умеет видеть перекрытия, а не только «один голос за раз».

На выходе — вероятность каждого класса по кадрам; берётся argmax. Затем срабатывает временная постобработка: `min_duration_on = 0.3 с` выбрасывает слишком короткие включения, `min_duration_off = 0.5 с` заклеивает паузы внутри одной реплики. Без этого шага на выходе был бы «дребезг» из сотен микросегментов.

Соседние окна перекрываются, поэтому один и тот же кадр получает несколько предсказаний. Они агрегируются (усреднение по перекрытию), и только после этого получается непрерывная разметка речи. Дальше она режется на кандидатов-сегменты — по смене активных спикеров и паузам.

6.2. Кандидаты-окна для эмбеддингов

Чтобы сравнивать голоса, нужны фрагменты, где говорит ровно один человек. В исследовательском пайплайне из разметки нарезаются окна двух масштабов: **2 с / 1 с** и **3 с / 1.5 с** (длина / шаг). Каждое окно проверяется двумя фильтрами:

- **purity ≥ 0.7** — доля доминирующего спикера в окне; окно на стыке двух голосов отбрасывается.
- **speech ≥ 50 %** — минимум половина окна должна быть речью, иначе эмбеддинг «размывается» тишиной.
- окна с перекрытием голосов не используются для обучения центроидов, но остаются в разметке.

На Android эти же окна формирует sherpa-onnx внутри `OfflineSpeakerDiarization`; снаружи доступны только параметры сегментации и кластеризации.

6.3. Эмбеддинги спикеров

Каждое чистое окно превращается в компактный вектор моделью ERes2Net base (37,8 МБ). В бенчмарке сравнивались также CAM++, TitaNet small и ERes2NetV2. Вектор L2-нормируется, поэтому близость считается косинусом:

```
cos(a, b) = (a · b) / (‖a‖ · ‖b‖), d(a, b) = 1 − cos(a, b)
```

Нормализация убирает влияние громкости и длины окна: важно направление вектора, а не его масштаб. Для двух окон одного человека косинус близок к 1, для разных — заметно ниже.

Модель	Размер	DER auto	DER oracle	RTF
CAM++ zh/en	28,3 МБ	33,20 %	27,69 %	0,094
ERes2Net en	26,5 МБ	33,43 %	27,28 %	0,202
TitaNet small	40,3 МБ	35,22 %	29,61 %	0,084
ERes2NetV2 zh	71,4 МБ	40,67 %	35,24 %	0,501

6.4. Кластеризация: AHC и fast clustering

Исследовательский пайплайн использует **complete-linkage AHC** по косинусной дистанции: на каждом шаге сливаются два кластера с минимальной максимальной дистанцией между их элементами.

```
d(A, B) = max d(a, b) для всех a из A и b из B
```

Complete-linkage не «размазывает» кластеры: если между двумя группами есть хотя бы одна далёкая пара, слияние откладывается. Для диаризации это важнее, чем average-linkage, потому что голоса двух людей могут быть близки в среднем, но иметь чёткую границу.

На Android кластеризацию делает sherpa-onnx (fast clustering) по порогу близости. Публичного экстрактора эмбеддингов в Kotlin-API нет, поэтому нельзя пересчитать матрицу расстояний снаружи и применить gap-статистику. Отсюда — второй способ выбора K.

6.5. Auto-K: два способа

Способ A — gap-статистика (offline, Python). Для K = 1…12 выполняется кластеризация и считается внутрикластерный разброс:

```
W(K) = сумма по кластерам r и точкам x внутри них от ‖x − mean(Cr)‖²
```

Затем строится эталон: данные равномерно заполняют bounding box, полученный через PCA, и повторяются 10 bootstrap-прогонов. Считается

```
gap(K) = E*[log W(K)] − log W(K)
```

Выбирается минимальное K, для которого gap(K) ≥ gap(K+1) − s(K+1), где s — стандартное отклонение bootstrap-прогонов. K = 1 — валидный случай: монолог не обязан делиться на спикеров.

Способ B — перебор порогов (Android). Порог кластеризации — единственная ручка, влияющая на K: низкий дробит спикеров, высокий склеивает всех в одного. Приложение делает 5 прогонов (0.4, 0.5, 0.6, 0.7, 0.8) и выбирает устойчивое плато:

- берётся K, встретившийся чаще всего;
- при равенстве частот — K, ближайший к медиане серии;
- внутри плато выбирается прогон, ближайший к середине диапазона порогов.

Порог	0,4	0,5	0,6	0,7	0,8
K	3	2	2	2	1
Итог		плато 0,5–0,7			выбран порог 0,6

Иллюстративный пример: чаще всего встречается K = 2, плато — 0,5–0,7, выбирается середина 0,6.

Почему на телефоне не gap: он требует матрицы расстояний и десятков прогонов, а Kotlin-API их не отдаёт. Sweep дешевле, детерминирован и объясним. Плата — нестабильность на длинных записях: на AMI фиксированный порог 0,5 даёт K = 8–22, и плато не образуется. На русских диалогах метод даёт 75 % точного K (RU dev), на подготовленном корпусе — 5/5.

6.6. Покадровое декодирование и склейка

После кластеризации нужно вернуть разметку во времени. Каждый кадр получает голосование окон: сколько окон относят его к тому или иному кластеру. Берётся top-K спикеров, затем сглаживание и те же `min_duration_on/off`. Результат — непрерывные сегменты (`start, end, speaker`), которые уже можно сравнивать со словами ASR.

Этап	Параметр	Значение
Сегментация	окно / сдвиг	10 с / 50 %
Сегментация	min on / min off	0,3 с / 0,5 с
Окна (research)	масштабы	2 с/1 с и 3 с/1,5 с
Окна (research)	purity / speech	≥ 0,7 / ≥ 50 %
Эмбеддинги	модель	ERes2Net base (APK)
Эмбеддинги	метрика	L2 + косинус
Кластеризация	research	complete-linkage AHC
Кластеризация	APK	fast clustering по порогу
Auto-K	research	gap, K = 1…12, bootstrap 10
Auto-K	APK	sweep 0,4–0,8, плато

6.7. Почему DER 18,97 % и 33,20 % — это не противоречие

33,20 % — micro-DER на смеси AMI + VoxConverse + RU. AMI — это дальние микрофоны и живые перекрытия, Vox — интервью в естественной среде, где спикеров может быть много. 18,97 % — тот же пайплайн на русских диалогах из двух голосов с чистой записью и паузами. Разные условия, разный уровень сложности; сравнивать эти числа напрямую нельзя.

Именно поэтому в отчёте обе цифры стоят рядом: 33,20 % показывает устойчивость на «грязном» миксе, 18,97 % — качество на целевом сценарии приложения.

7. Назначение спикера каждому слову

Для каждого слова считается временное перекрытие с каждым сегментом диаризации. Побеждает сегмент с максимальным перекрытием:

```
overlap = min(word.end, seg.end) − max(word.start, seg.start)
```

Если слово попало в паузу и перекрытия нет, берётся ближайший по времени сегмент. Правило детерминированное и не делает лишних проходов по аудио. В Python-пайплайне на стыках реплик дополнительно проверялись эмбеддинги слова, но на Android экстрактор недоступен — осознанный компромисс.

Точность такого назначения на RU test: 82,8 % при auto-K и 90,3 % при oracle-K (484 слова). Ошибки сосредоточены на коротких словах в местах смены говорящего.

8. Склейка реплик и JSON

Слова собираются в реплики по трём правилам. Новая реплика начинается, если:

- сменился спикер;
- пауза между словами больше 1,2 с;
- текущая реплика уже длиннее 15 с (длинный монолог режется для LLM и чтения).

На выходе — ровно тот JSON, который ждёт заказчик:

```
{"transcript":[{"speaker_id":1,"start_ms":160,"end_ms":3640,"text":"Ну что ж, будем дальше
исследовать мир китайской кухни."}]}
```

`speaker_id` — целое число от 1 до K. Нумерация устойчивая: первый появившийся спикер получает 1. Экспорт идёт через системный диалог сохранения файла, поэтому JSON можно положить в любое место на устройстве.

9. Локальная LLM и системный промпт

Пунктуацию и регистр правит **Qwen2.5 1.5B Instruct Q4_K_M** — квантованная GGUF-модель на 1,04 ГБ, запущенная через llama.cpp на CPU. Она не переписывает смысл: на вход даётся одна реплика, на выход — та же реплика с пунктуацией.

Параметры генерации

Параметр	Значение	Почему
Контекст	2048 токенов	хватает на реплику + промпт
Потоки	4	баланс скорости и нагрева
temperature	0.0	воспроизводимость
top_p / top_k	1.0 / 1	жадная генерация
seed	42	одинаковый результат
max_tokens	128…512	зависит от длины реплики

Промпт хранится в SharedPreferences и редактируется прямо на главном экране: карточка «Промпт LLM» → «Изменить». Это позволяет менять поведение (например, «сохраняй разговорную лексику» или «пиши цифры словами») без пересборки APK. Кнопка «Сбросить» возвращает заводскую инструкцию.

Заводской промпт

> *«Ты — редактор расшифровки русской речи. На вход даётся одна реплика. Расставь знаки препинания, заглавные буквы и исправь очевидные ошибки распознавания. Не добавляй новую информацию, не меняй смысл и не пересказывай. Верни ТОЛЬКО исправленный текст реплики, без кавычек, пояснений и markdown.»*

Страховка

LLM может вернуть пустоту, мусор или текст без пунктуации. Тогда включается `LocalTextPolish`: убирает кавычки по краям, схлопывает пробелы, поднимает первую букву, поднимает регистр после `. ! ? …` и добавляет точку в конце. Текст никогда не остаётся «потоком слов». Слова и тайминги при этом не меняются — правка касается только строки текста.

Стоимость

1.5B Q4_K_M занимает ~1,4–1,6 ГБ RSS и добавляет примерно 25–40 секунд на 56-секундный диалог из 4 реплик (замер на iQOO 15R). Именно поэтому полоса прогресса после диаризации идёт медленно: это не зависание, а генерация по каждой реплике.

Платформенное ограничение

Бесплатный AAR `llama-android` содержит только `arm64-v8a`. На x86_64-эмуляторе LLM не загружается, и приложение автоматически отключает автоскачивание, оставляя rule-based fallback. Для реального устройства нужен arm64.

10. Метрики и результаты

Что означают цифры

Метрика	Смысл
WER	доля ошибок в словах: (S+D+I)/N, меньше — лучше
CER	то же по символам
DER	(confusion + miss + false alarm) / длительность речи
K exact	доля файлов, где число спикеров угадано точно
K MAE	средняя абсолютная ошибка в числе спикеров
Word accuracy	доля слов с правильно назначенным спикером
RTF	время обработки / длительность аудио; < 1 — быстрее реального времени

ASR: FLEURS ru + Golos crowd, 200+200 фраз

Модель	WER avg	RTF	RAM
GigaAM v3	**2,44 %**	0,043	337 МБ
Zipformer-RU	3,55 %	0,019	401 МБ
Parakeet TDT 0.6B	7,39 %	0,064	988 МБ
Whisper large-v3-turbo	18,18 %	0,382	2288 МБ

Диаризация: 1 AMI + 1 Vox + 8 RU, DER усреднён по трём датасетам

Режим	Модель	DER	K exact
auto K	CAM++ zh/en	33,20 %	50 %
oracle K	ERes2Net en	27,28 %	—
sweep (Kotlin)	ERes2Net base	27,85 % (RU dev)	75 %

Полный пайплайн на русских диалогах (5 файлов, ~8,3 мин)

Режим	DER	Word accuracy	K exact
auto K	**18,97 %**	**88,11 %**	5/5
oracle K	18,97 %	88,11 %	5/5

DER 18,97 % на диалогах заметно лучше 33 %, потому что там чистый русский материал, два спикера и нормальные паузы. Разброс по файлам большой: 5,1–46,1 % — один диалог с перекрытиями и короткими репликами заметно сложнее.

Тесты

71 JVM-тест на ASR, 25 на диаризацию и 63 на Android-приложение — всего **159 тестов**, 0 падений. Отдельно проверены формат JSON (целочисленный speaker_id), fallback без LLM и целостность файлов моделей.

11. Проверка на реальном устройстве

iQOO 15R (Android 16, arm64-v8a), диалог `dialogs_ru_002.wav` из датасета: 56,5 с, два спикера. Модели заранее залиты в filesDir, чтобы проверять пайплайн, а не сеть.

Показатель	Результат
Длительность	56,5 с
Спикеров	2 (K угадан)
Слов	119
Реплик	4
Пунктуация	корректная, от Qwen2.5
Время распознавания	≈ 50–60 с вместе с LLM

Границы реплик: приложение против разметки

Спикер	Приложение	Разметка	Расхождение
1	0,16–3,64	0,00–4,23	0,16 / 0,59 с
2	4,76–18,14	4,68–19,16	0,08 / 1,02 с
1	19,84–35,26	19,61–35,32	0,23 / 0,06 с
2	36,02–55,80	35,77–56,03	0,25 / 0,23 с

Расхождение — по началу и концу сегмента; смена говорящего определена верно во всех случаях.

12. Ограничения

- LLM работает только на arm64: бесплатный AAR llama.cpp не содержит x86_64.
- Модель LLM весит 1,04 ГБ и требует интернета при первом запуске.
- Auto-K через sweep нестабилен на длинных записях: на AMI плато не образуется.
- Перекрытия речи обрабатываются частично: слово получает одного спикера.
- Ресемплинг линейный — для музыки и широкополосного шума не оптимален.
- Русский — приоритет: на английском WER у GigaAM заметно выше.
- Нет экспорта RTTM и пакетной обработки очереди файлов.

Все ограничения продублированы в docs/LIMITATIONS.md, методика — в docs/PROTOCOL.md, цифры — в docs/TEST_RESULTS.md.

13. Где что лежит в коде

Шаг	Файл
Декодирование	com/nuclearhack/transcription/AudioDecoder.kt
Нарезка	com/nuclearhack/transcription/AudioChunker.kt
ASR	com/nuclearhack/transcription/GigaAmTranscriber.kt
Токены → слова	com/nuclearhack/transcription/WordTimestamps.kt
Диаризация	com/nuclearhack/diarization/SherpaDiarizer.kt

Auto-K	com/nuclearhack/diarization/SpeakerCountSweep.kt
Спикер словам	com/nuclearhack/diarization/WordSpeakerAssigner.kt
Реплики	com/nuclearhack/diarization/UtteranceBuilder.kt
LLM	com/nuclearhack/transcription/LlamaCppLocalLlm.kt
Промпт	com/example/baseapp/transcriber/LlmPromptStore.kt
Оркестратор	com/example/baseapp/transcriber/GigaAmTranscriber.kt

Отчёт описывает сборку из коммита main на 8 ноября 2026: APK 93 863 438 байт, SHA-256 8107B440…A406.